FIAP

Data Structure and Algorithms

Ciência da Computação

Prof. Érick T. Yamamoto

Syllabus - Git e GitHub com C usando VS Code

O que vocês irão aprender nesta aula:

- criar um projeto em C no VS Code;
- inicializar Git;
- fazer commits;
- publicar no GitHub;
- atualizar e versionar o projeto sem sair do editor.

Problema!

Imagine que você está programando em C.

Depois de várias alterações o programa para de funcionar.

Como voltar para a versão que funcionava ontem?

Git resolve exatamente esse problema.

O que é Git?

Git é um sistema de controle de versão.



git

Ele registra:

1. **histórico do código**
2. **quem alterou**
3. **quando alterou**
4. **o que mudou**

O que é GitHub?

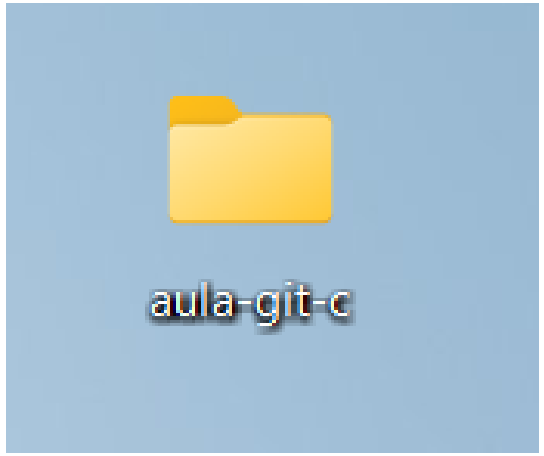
GitHub é uma plataforma online para hospedar repositórios Git.



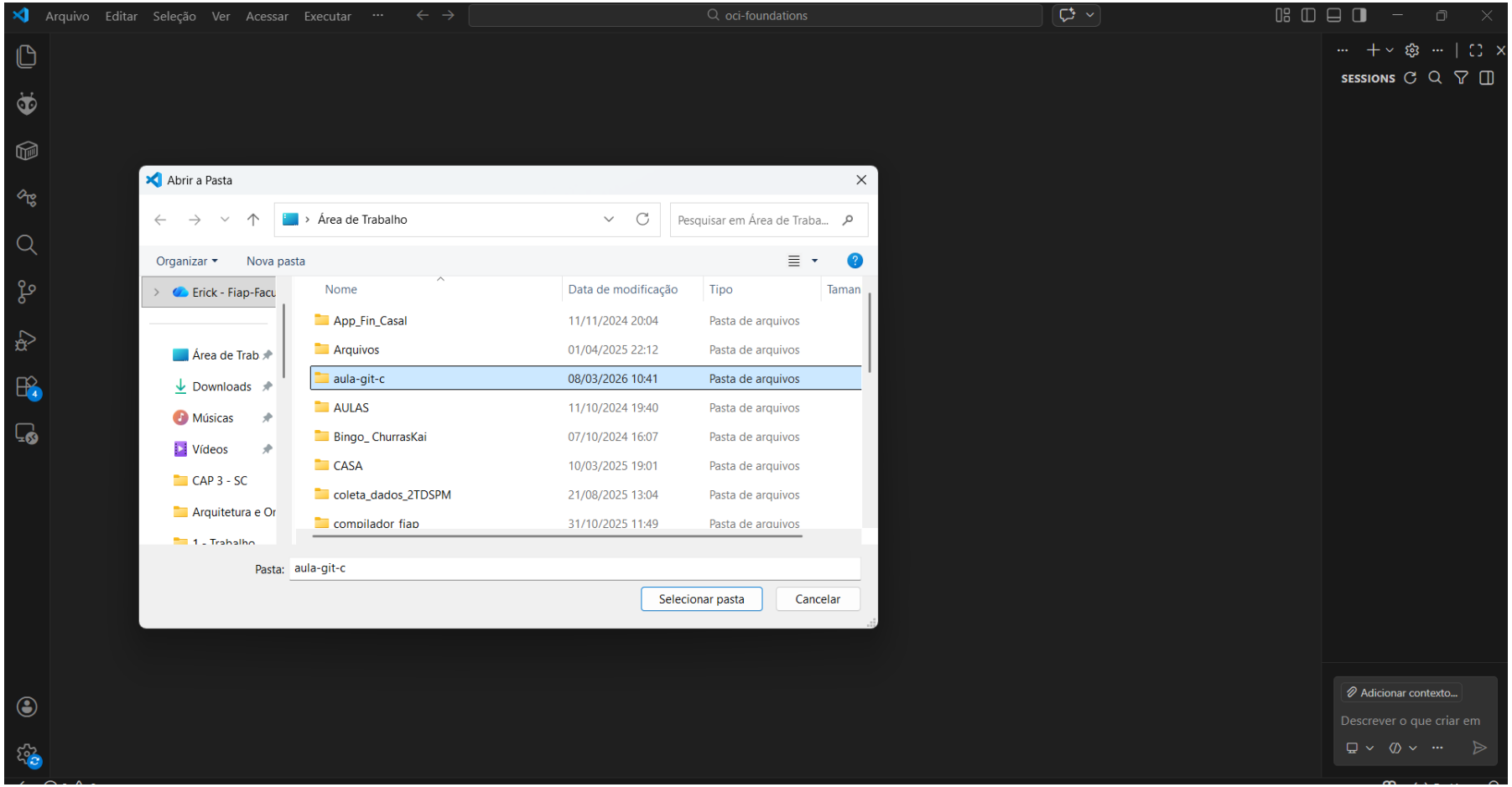
Criada em October 19, 2007. O mascote do GitHub é Mona, uma "gata polvo" antropomorfizada com cinco braços semelhantes aos de um polvo. A personagem foi criada pelo designer gráfico Simon Oxley como uma imagem clipart para vender no iStock.

Criando o projeto - Programa em C

Criar uma pasta aula-git-c

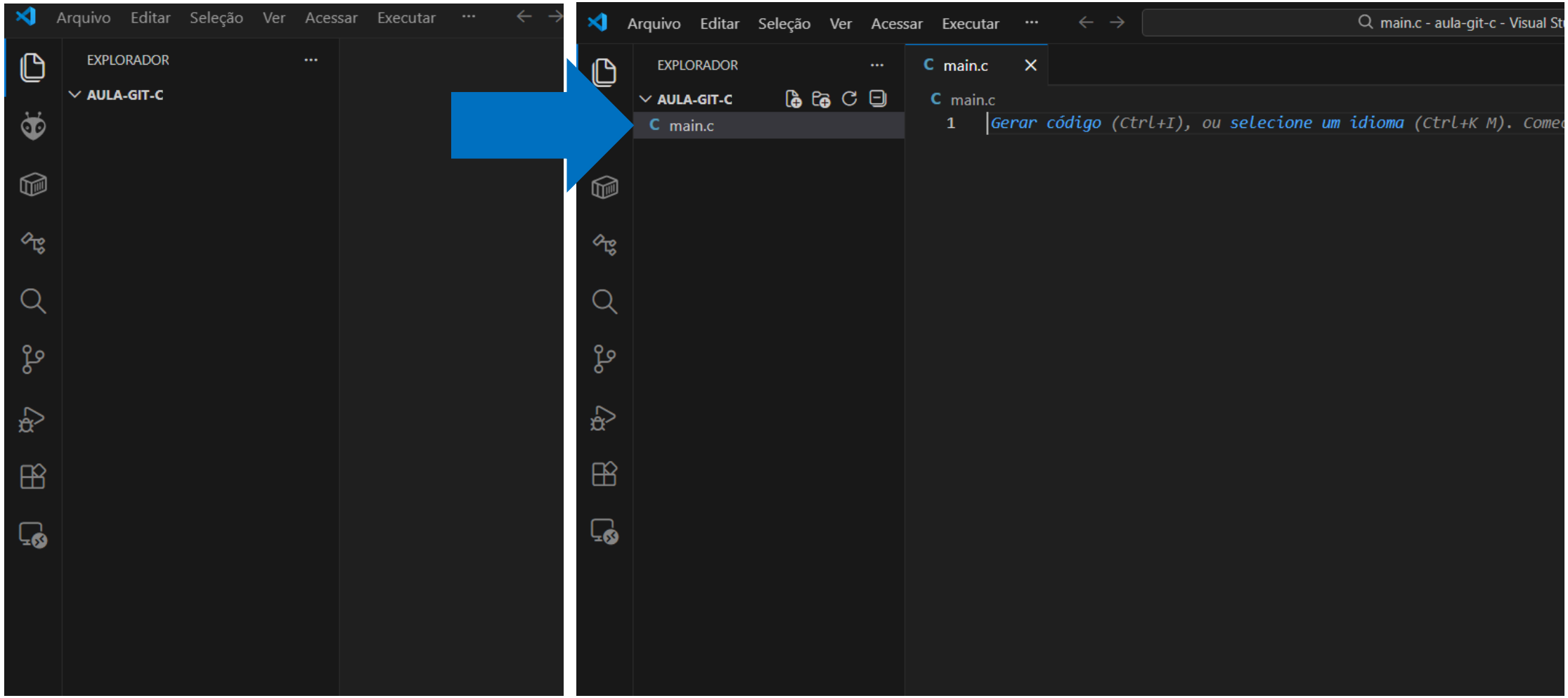


Selecione a pasta criada dentro do VSCode



Criando o projeto - Programa em C

Dentro da Pasta, crie um arquivo com o nome **main.c**



Criando o projeto - Programa em C

Dentro do arquivo **main.c**, escreva as seguintes linhas de código:

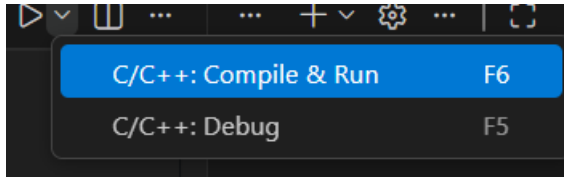
main.c

```c
#include <stdio.h>

int main(){
    printf("Hello Git no VS Code!\n");
    return 0;
}
```

Não esqueça de sempre salvar o seu código!

Agora iremos, compilar e rodar o código!



Ou pelo terminal:

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS

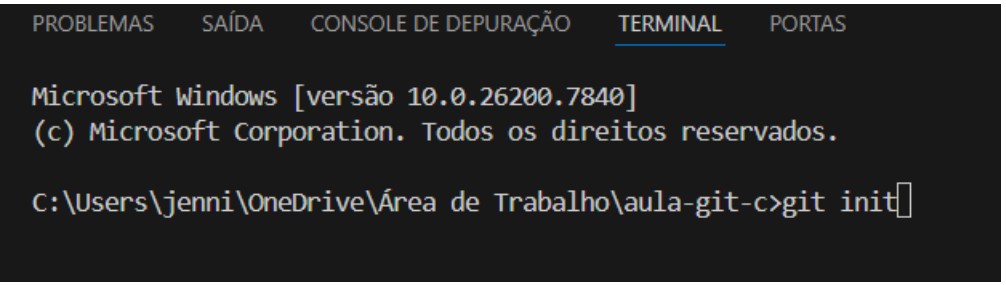
PS C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c> gcc main.c -o programa
```

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS

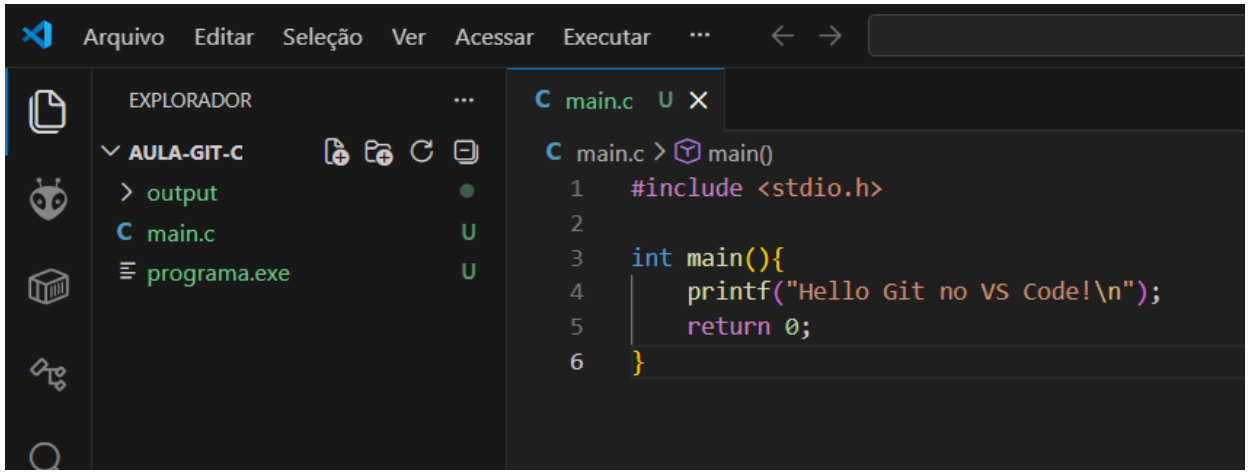
PS C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c> ./programa
Hello Git no VS Code!
PS C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>
```

Criando o projeto - Configuração Git e GitHub

Dentro do Terminal, para inicializarmos o Git é necessário digitar o seguinte comando:



Agora iremos, compilar e rodar o código!



Agora o Git começa a acompanhar as mudanças do projeto.

Criando o projeto - Configuração Git e GitHub

Vamos verificar o Status do nosso projeto?
Digite no terminal → **git status**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git status
```



```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        main.c
        output/
        programa.exe

nothing added to commit but untracked files present (use "git add" to track)
```

Pode reparar que o Git, te solicita a inclusão dos arquivos da pasta. Então vamos fazer isso!

Criando o projeto - Configuração Git e GitHub

Vamos verificar o Status do nosso projeto?
Digite no terminal → **git status**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git status
```



```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        main.c
        output/
        programa.exe

nothing added to commit but untracked files present (use "git add" to track)
```

Pode reparar que o Git, te solicita a inclusão dos arquivos da pasta. Então vamos fazer isso!

Criando o projeto - Configuração Git e GitHub

Para adicionar um arquivo ao controle de versão:
Digite no terminal → **git add "NOME DO ARQUIVO"**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git add main.c
```

Ou para adicionar todos arquivos.
Digite no terminal → **git add .**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git add .
```

Digite novamente no terminal → **git status**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   main.c
        new file:   output/main.exe
        new file:   programa.exe
```

Podemos verificar que os arquivos foram incluídos com sucesso!

Criando o projeto – Configuração Git e GitHub

Agora vamos criar o nosso primeiro commit.
Digite no terminal → **git commit -m “ADICIONAR NOVO RECURSO”**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git commit -m "Primeiro programa em C"
```

Use esse comando para criar uma mensagem de commit para as alterações, tornando-as parte do histórico do seu projeto.

```
[master (root-commit) 2dc0283] Primeiro programa em C
 3 files changed, 6 insertions(+)
 create mode 100644 main.c
 create mode 100644 output/main.exe
 create mode 100644 programa.exe
```

3 files changed → 3 arquivos foram incluídos ou modificados no commit
6 insertions(+) → 6 linhas de código foram adicionadas

Isso significa que o Git **criou e passou a versionar esses arquivos** no repositório.

O ideal é versionar apenas:

1. **Código fonte (.c);**
2. **Documentação;**
3. **arquivos de configuração.**

Criando o projeto – Configuração Git e GitHub

E os arquivos .exe ou algum arquivo que queira ignorar ou não incluir?

Na lateral esquerda do VS Code:

1. Clique com o botão direito na pasta do projeto
2. Clique em **New File**
3. Digite o nome:



Depois abra o arquivo e adicione:



Não esquecer de salvar o arquivo, adicionar e fazer o *commit.*

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git add .gitignore

C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git commit -m "Adiciona o arquivo .gitignore"
[master a78d1ca] Adiciona o arquivo .gitignore
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
```

Criando o projeto – Configuração Git e GitHub

E os arquivos que já estão commitados/ versionado?

O .gitignore **não remove arquivos que já foram versionados**.
Como seus .exe já entraram no commit, vamos removê-los do controle do Git:

Digite no terminal → **git rm --cached *.exe**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git rm --cached *.exe
rm 'output/main.exe'
rm 'programa.exe'
```

Não esquecer de "*commitar*", o que foi feito. SEMPRE!

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git commit -m "Remove executáveis do repositório"
[master bb9e0b5] Remove executáveis do repositório
 2 files changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 output/main.exe
 delete mode 100644 programa.exe
```

Agora vamos fazer o mesmo para a pasta *output!*

Criando o projeto – Configuração Git e GitHub

E aí conseguiu?

Agora vamos fazer o mesmo para a pasta *output!*

Apareceu a seguinte mensagem?

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git rm --cached output
fatal: pathspec 'output' did not match any files
```

Para removermos a pasta é um comando direto no terminal pelo comando a seguir

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>rmdir /s /q output
```

Ou deletando a pasta e iniciar todo comando do git desde a sua inicialização.

Criando o projeto - Configuração Git e GitHub

Agora vamos verificar os históricos dos *commits:*

No terminal digite → **git log**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git log
commit bb9e0b52f48e924acb4bad16b35771c74739c213 (HEAD -> master)
Author: ErickTYamamoto <ety.ufabc@gmail.com>
Date:   Mon Mar 9 08:41:04 2026 -0300

    Remove executáveis do repositório

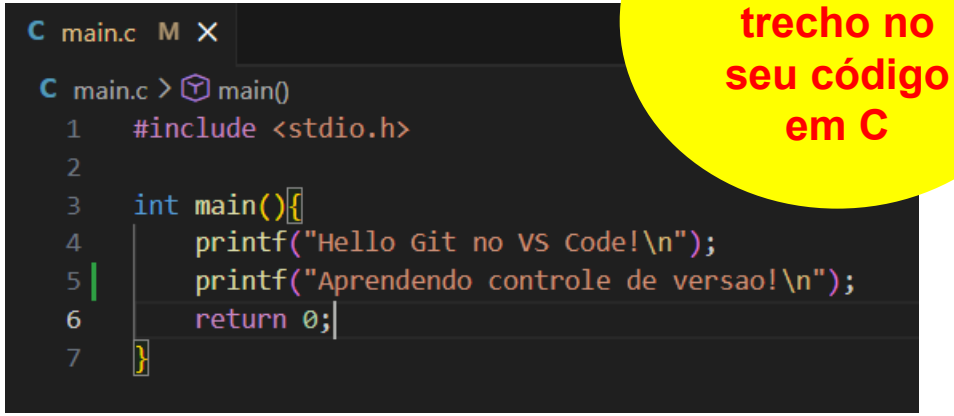
commit a78d1caf7b90b2e002adfce8d2f5d0864406366a
Author: ErickTYamamoto <ety.ufabc@gmail.com>
Date:   Mon Mar 9 08:34:01 2026 -0300

    Adiciona o arquivo .gitignore

commit 2dc028391a54e41f3e2b82510569f1535fa758b3
Author: ErickTYamamoto <ety.ufabc@gmail.com>
Date:   Mon Mar 9 08:14:02 2026 -0300

    Primeiro programa em C
```

Adicione o seguinte trecho no seu código em C



Criando o projeto - Configuração Git e GitHub

O que mudou?

Adicione o seguinte trecho no seu código em C

main.c M

```c
#include <stdio.h>

int main(){
    printf("Hello Git no VS Code!\n");
    printf("Aprendendo controle de versao!\n");
    return 0;
}
```

No terminal digite → **git diff**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git diff
diff --git a/main.c b/main.c
index f693398..c55693e 100644
--- a/main.c
+++ b/main.c
@@ -2,5 +2,6 @@

 int main(){
     printf("Hello Git no VS Code!\n");
+    printf("Aprendendo controle de versao!\n");
     return 0;
 }
\ No newline at end of file
```

Não esqueçam de *commitar:*

git add .

git commit -m "Adiciona segunda mensagem"

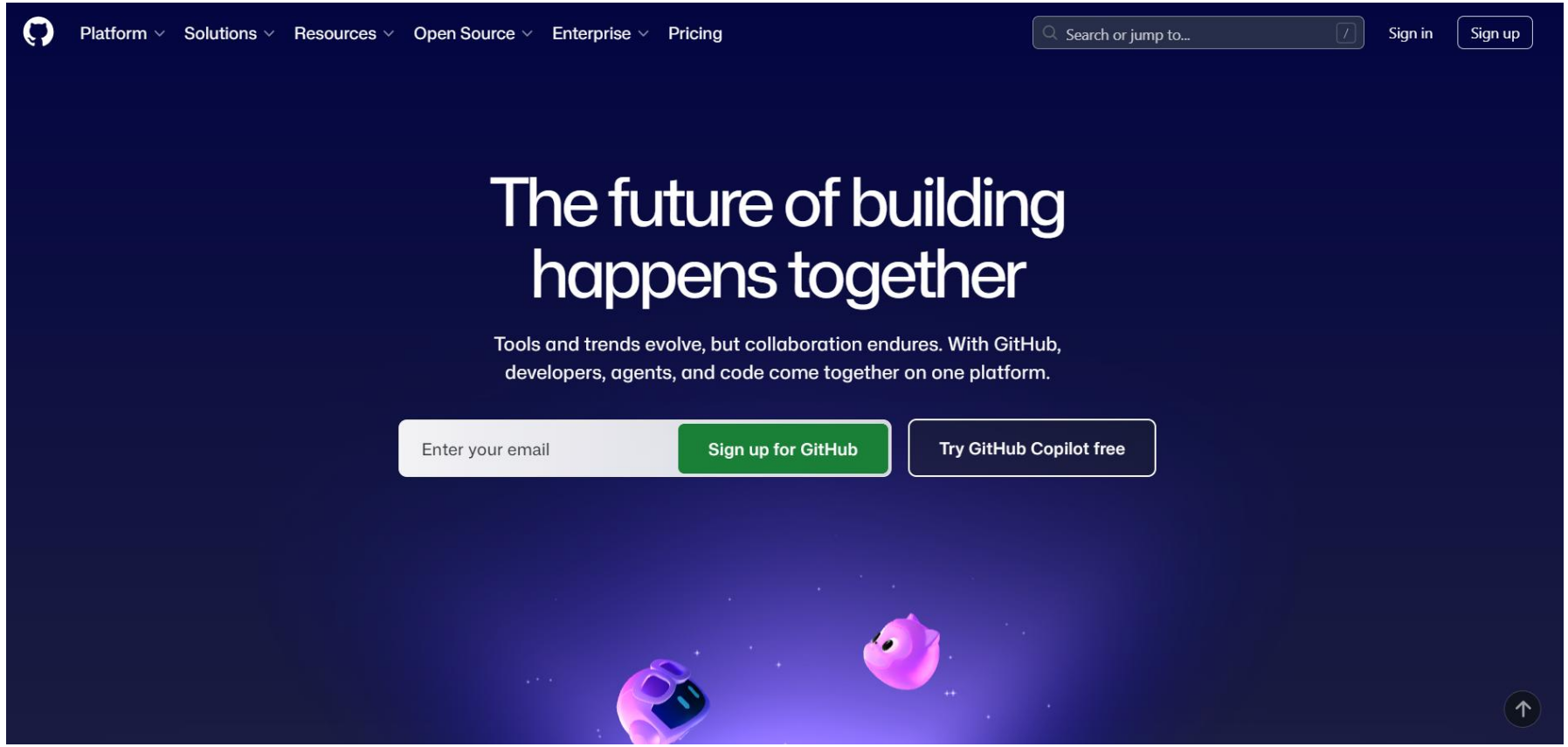
```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git add .

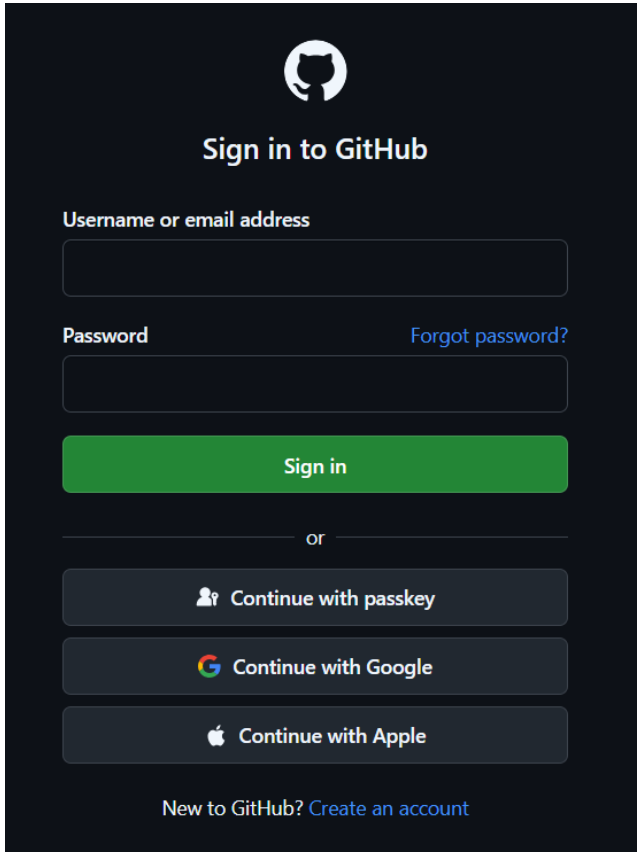
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git commit -m "Adiciona segunda mensagem"
[master 9491270] Adiciona segunda mensagem
 1 file changed, 1 insertion(+)
```

Criando o projeto - Configuração Git e GitHub

Vamos mandar para o seu GitHub:

No seu navegador de preferência entre em: https://github.com/



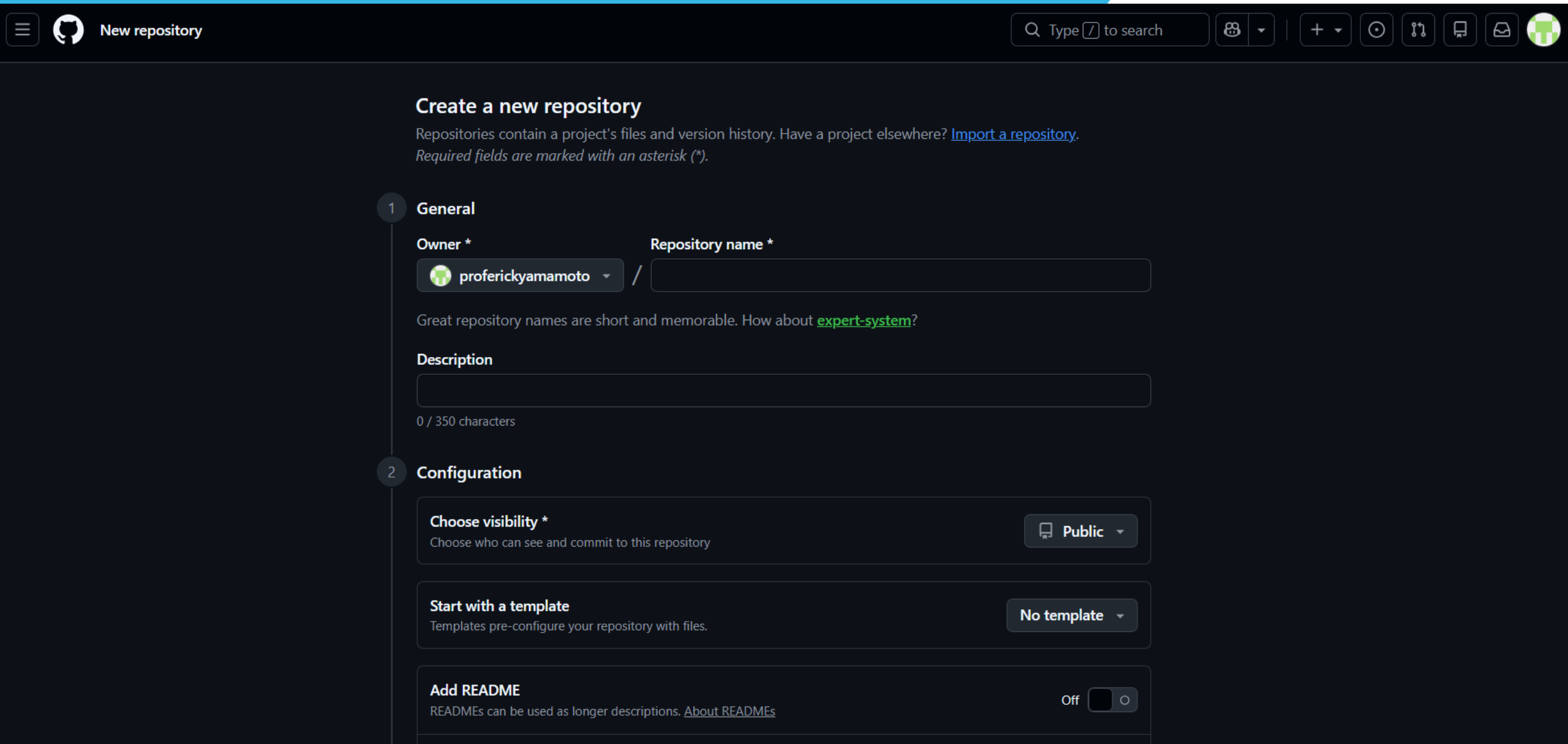


Criando o projeto - Configuração Git e GitHub

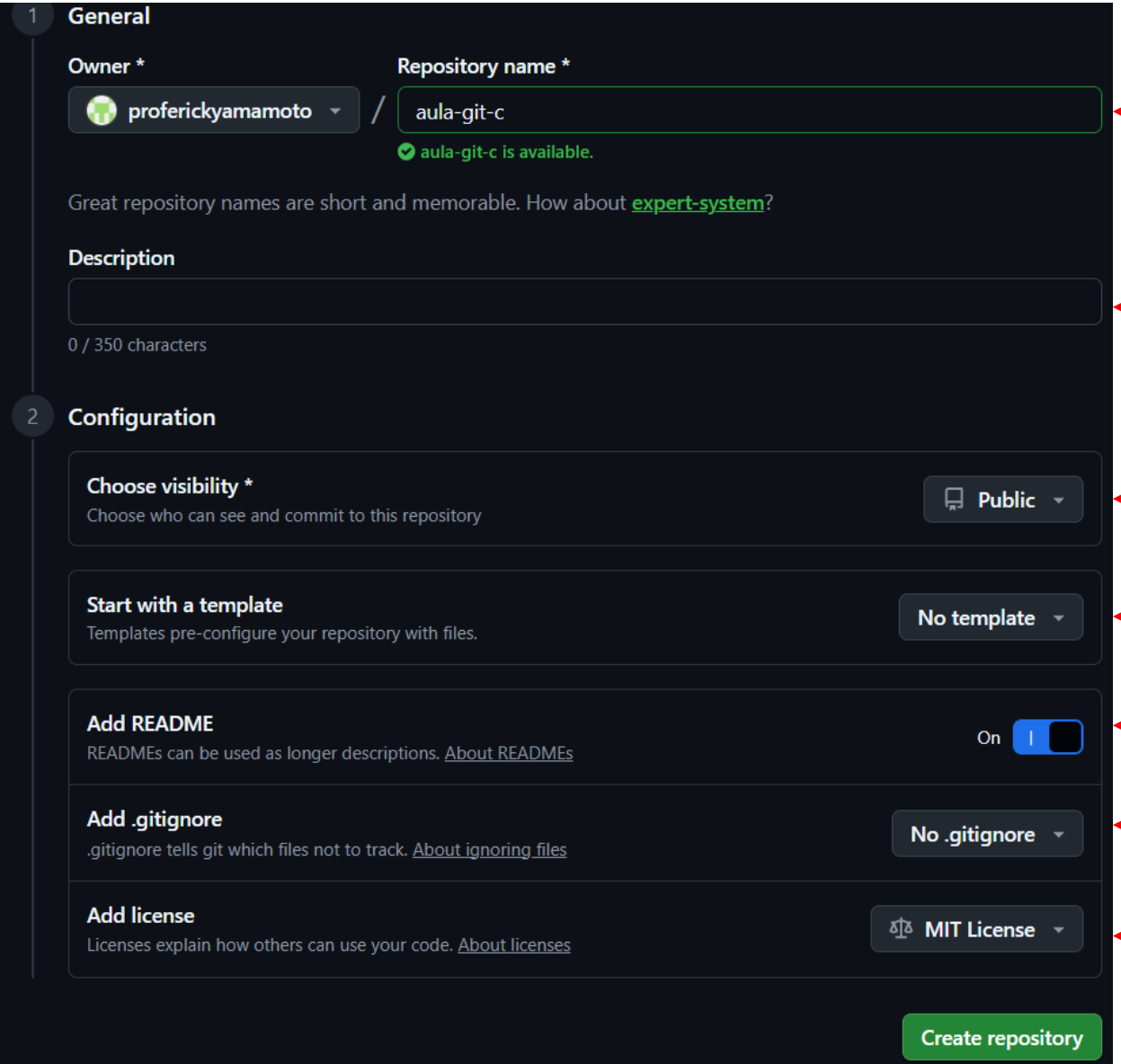


Aqui vocês terão o histórico do repositório.

Criando o projeto - Configuração Git e GitHub



Criando o projeto - Configuração Git e GitHub



Nome do Repositório

Descrição do Repositório

Visibilidade do seu Repositório

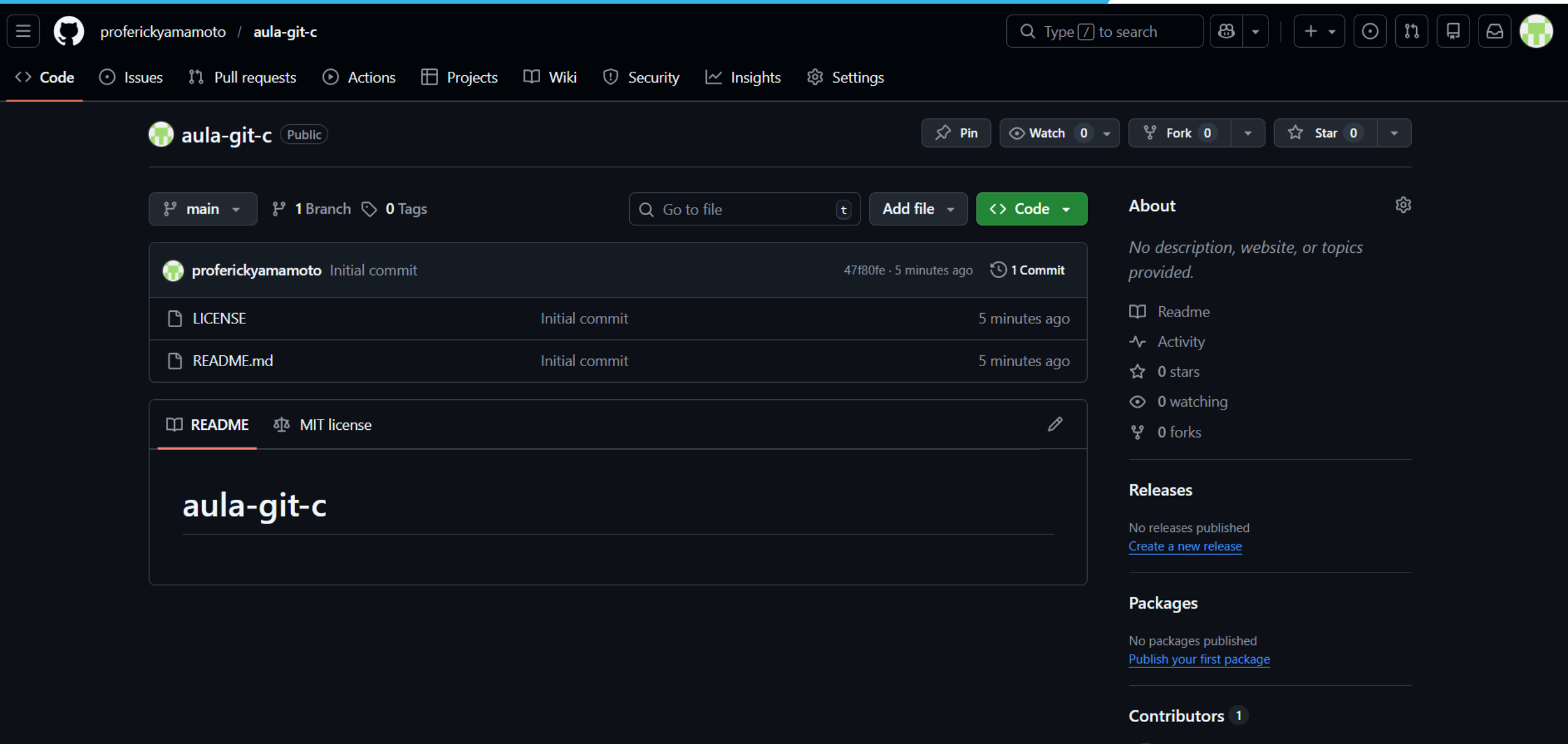
Se quiser um template pronto do seu repositório
Ou de outro usuário

Adicionar um README

Adicionar um .gitignore, fizemos no nosso projeto

Licença para cópia, sempre é bom deixar
Selecionado um, recomendo o do MIT License

Criando o projeto - Configuração Git e GitHub



Criando o projeto – Configuração Git e GitHub

Vamos conectar o projeto ao GitHub:

Voltando ao projeto, digite no terminal → **git branch -M main**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git status
On branch main
nothing to commit, working tree clean
```

A Branch é uma linha de desenvolvimento paralela e independente, funcionando como uma cópia de trabalho isolada do seu projeto principal (geralmente chamado de main ou master)

Agora vamos acessar o seu GitHub pelo terminal, digite no terminal → **git remote add origin https://github.com/USUARIO/aula-git-c.git**

Aqui coloque o seu usuário do GitHub. Vai estar do lado da Mona.

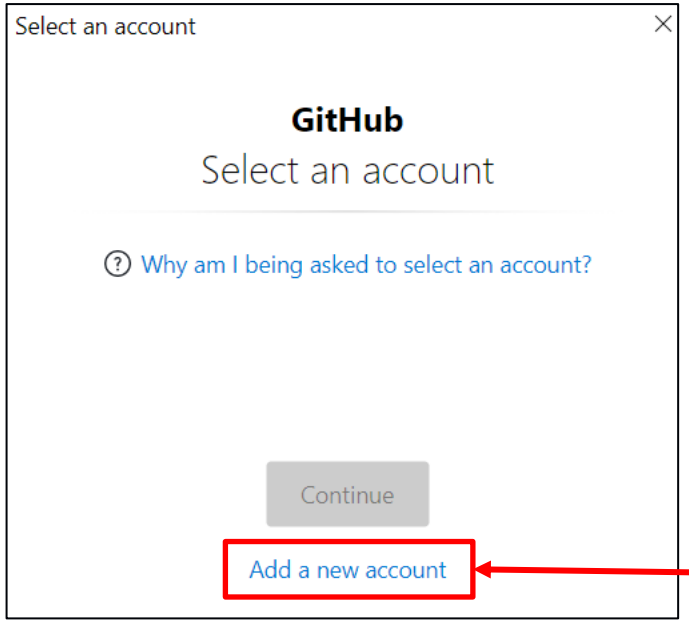
Criando o projeto - Configuração Git e GitHub

Vamos conectar o projeto ao GitHub:

Conectado, vamos enviar ao repositório online. Digite no terminal → **git push -u origin main**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git push -u origin main
info: please complete authentication in your browser...
```

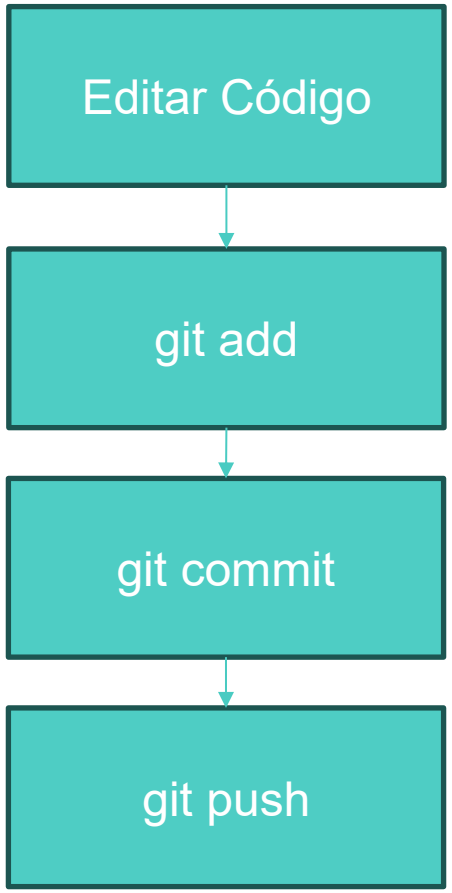
Será solicitado a autenticação, o Push envia o código para o repositório online.



Clique aqui e adicione sua conta do GitHub

Criando o projeto - Configuração Git e GitHub

Um Fluxograma do GitHub Básico:



Criando o projeto - Configuração Git e GitHub

Vamos conectar o projeto ao GitHub:

Todas as conexões e autenticações feitas. Digite no terminal → **git pull origin main**

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.45 KiB | 54.00 KiB/s, done.
From https://github.com/proferickyamamoto/aula-git-c
 * [new branch]      main       -> origin/main
There is no tracking information for the current branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

    git pull <remote> <branch>

If you wish to set tracking information for this branch you can do so with:

    git branch --set-upstream-to=origin/<branch> main
```

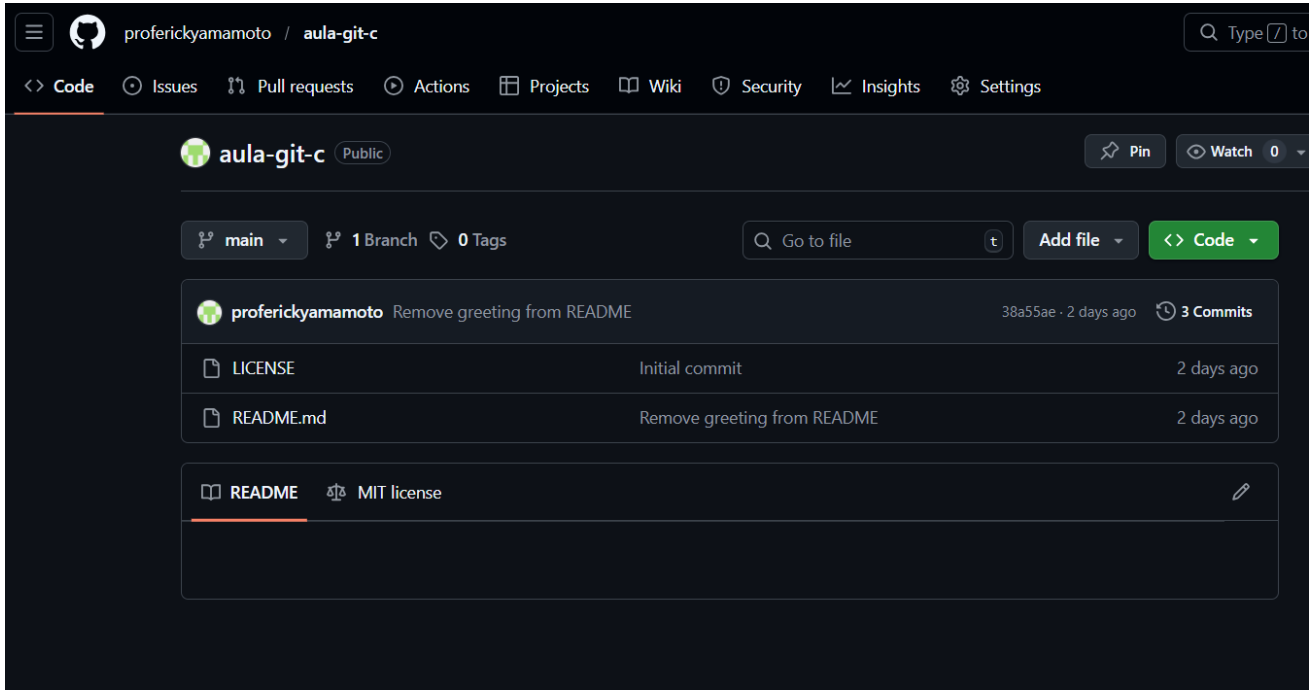
Caso não tenha o repositório, aparecerá:

```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git pull
remote: Repository not found.
fatal: repository 'https://github.com/proferickyamamoto/aula-git-c.git/' not found
```

Criando o projeto - Configuração Git e GitHub

Vamos conectar o projeto ao GitHub:

Caso você veja alguns arquivos que não tem no seu projeto. Digite no terminal → **git pull origin main --allow-unrelated-histories**



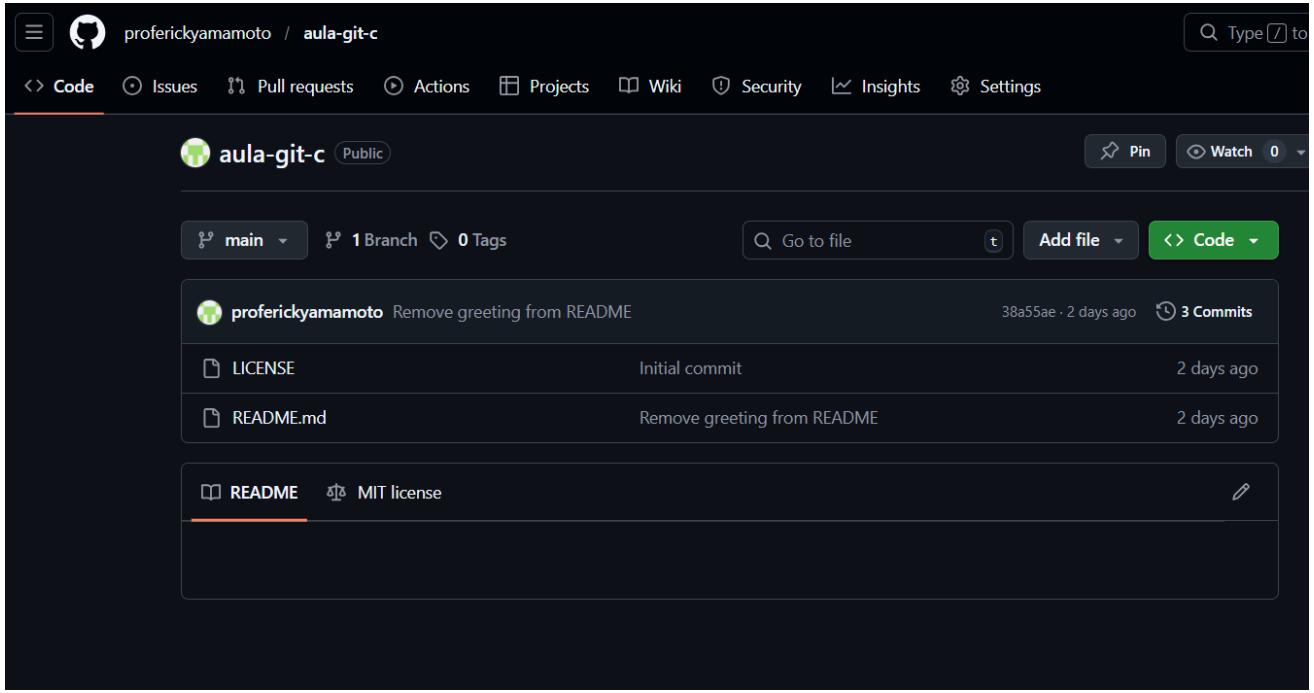
Reparem que nem o LICENSE enem o README.md estão no projeto do VSCode. Com o comando acima, os arquivos irão para o seu projeto



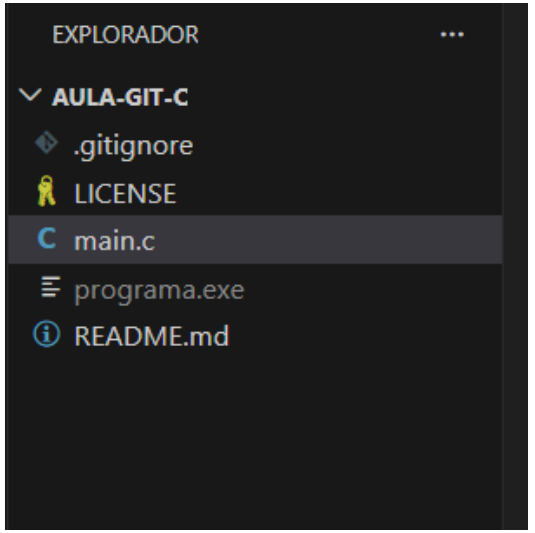
Criando o projeto - Configuração Git e GitHub

Vamos conectar o projeto ao GitHub:

O resultado segue abaixo:



Agora sim, estão iguais tanto o GitHub e o projeto.



Criando o projeto - Configuração Git e GitHub

Vamos conectar o projeto ao GitHub:

Agora possibilidade mandar e trazer sem problemas. Ou seja, podemos fazer o comando **push** e **pull**.

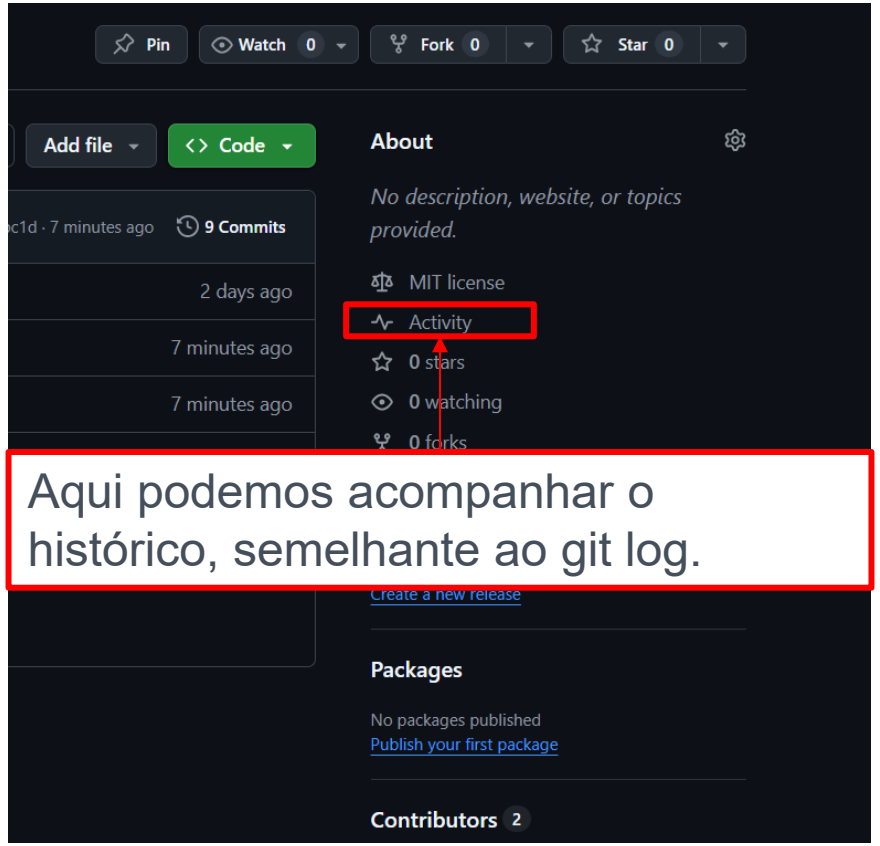
```
C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git commit -m "Adicionado arquivos de licença e README"
[main d9cbc1d] Adicionado arquivos de licença e README
 2 files changed, 1 insertion(+), 2 deletions(-)

C:\Users\jenni\OneDrive\Área de Trabalho\aula-git-c>git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 410 bytes | 410.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/proferickyamamoto/aula-git-c.git
   ed9284b..d9cbc1d  main -> main
```

O resultado do push fica igual ao lado no terminal

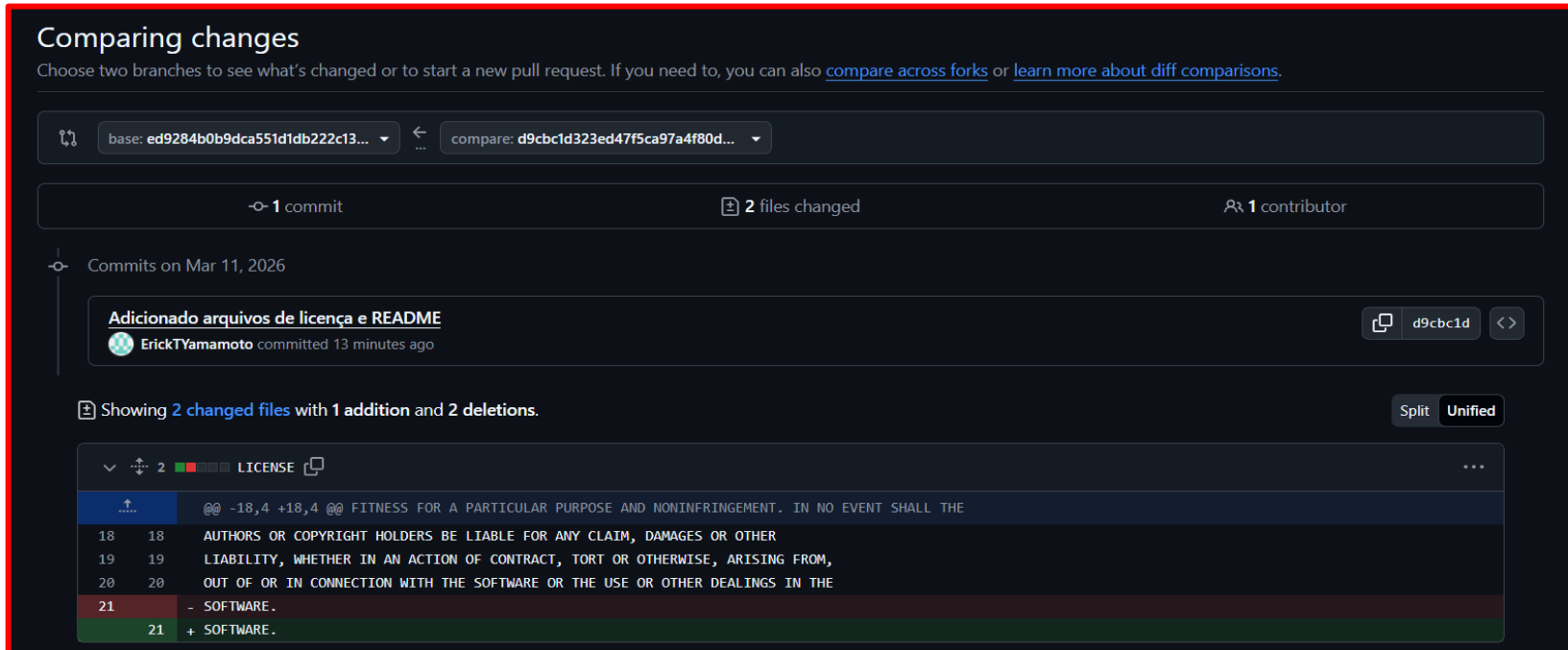
Criando o projeto - Configuração Git e GitHub

Agora conseguimos acompanhar no GitHub também.



Aqui podemos acompanhar o histórico, semelhante ao git log.






DÚVIDAS?!
OBRIGADO!!

Copyright © 2026
Prof. Dr. Erick T. Yamamoto.
Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor (autor).